

TÉCNICAS DE APRENDIZADO DE MÁQUINA

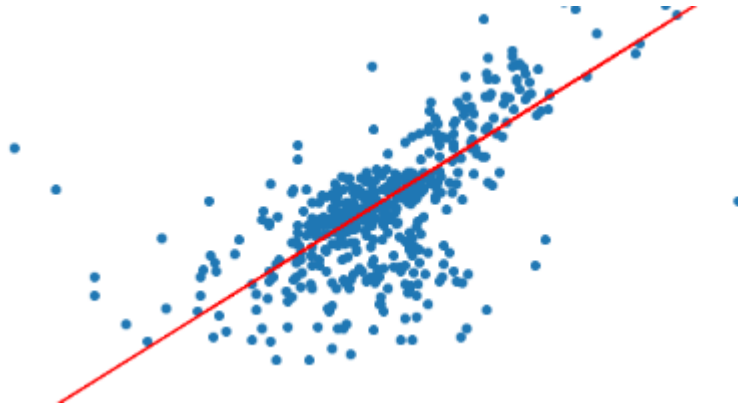
Bacharelado em Sistemas da Informação

Prof. Marco André Abud Kappel

Aula 12 – Regressão com SVM

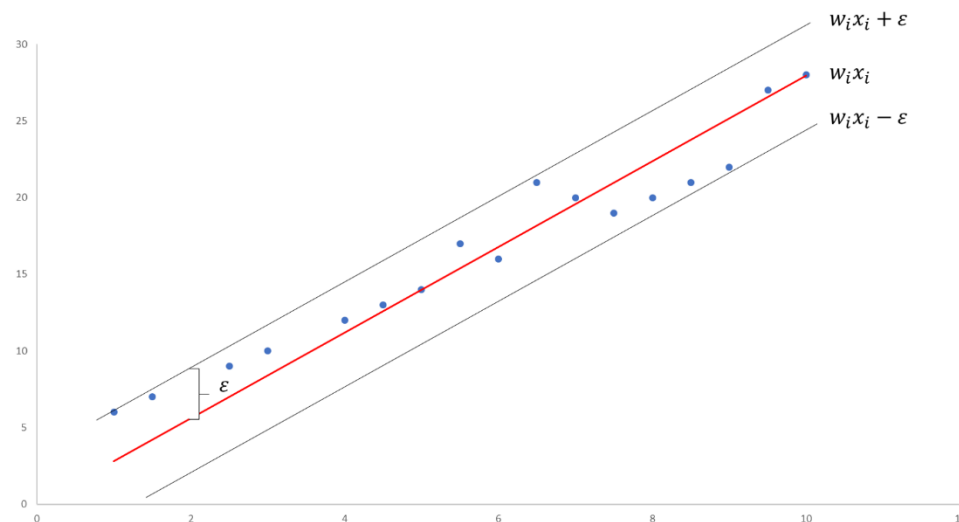
Regressão com SVM

- **Support Vector Regression (SVR)**
 - As *Support Vector Machines* (SVM) são muito conhecidas pela sua aplicação em problemas de classificação.
 - Porém, também é possível utilizar esta técnica em problemas de **regressão**.
 - Nestes casos, a técnica passa a ser conhecida como *Support Vector Regression* (SVR).



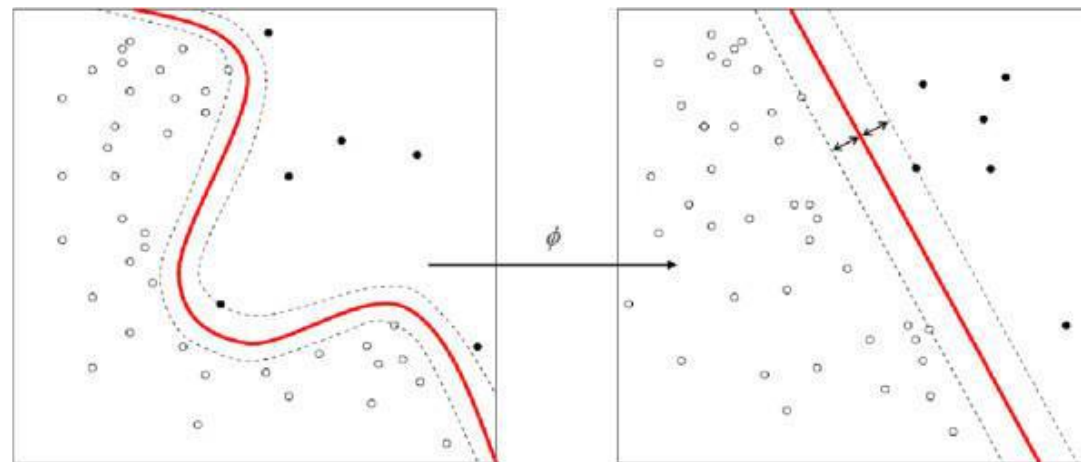
Regressão com SVM

- **Support Vector Regression (SVR)**
 - A utilização do SVR permite que seja definida uma margem de erro “ ϵ ” que será considerada aceitável na modelagem.
 - Com isso, ele tentará encontrar a **melhor reta** (hiperplano em maiores dimensões) que se **ajuste** aos dados.



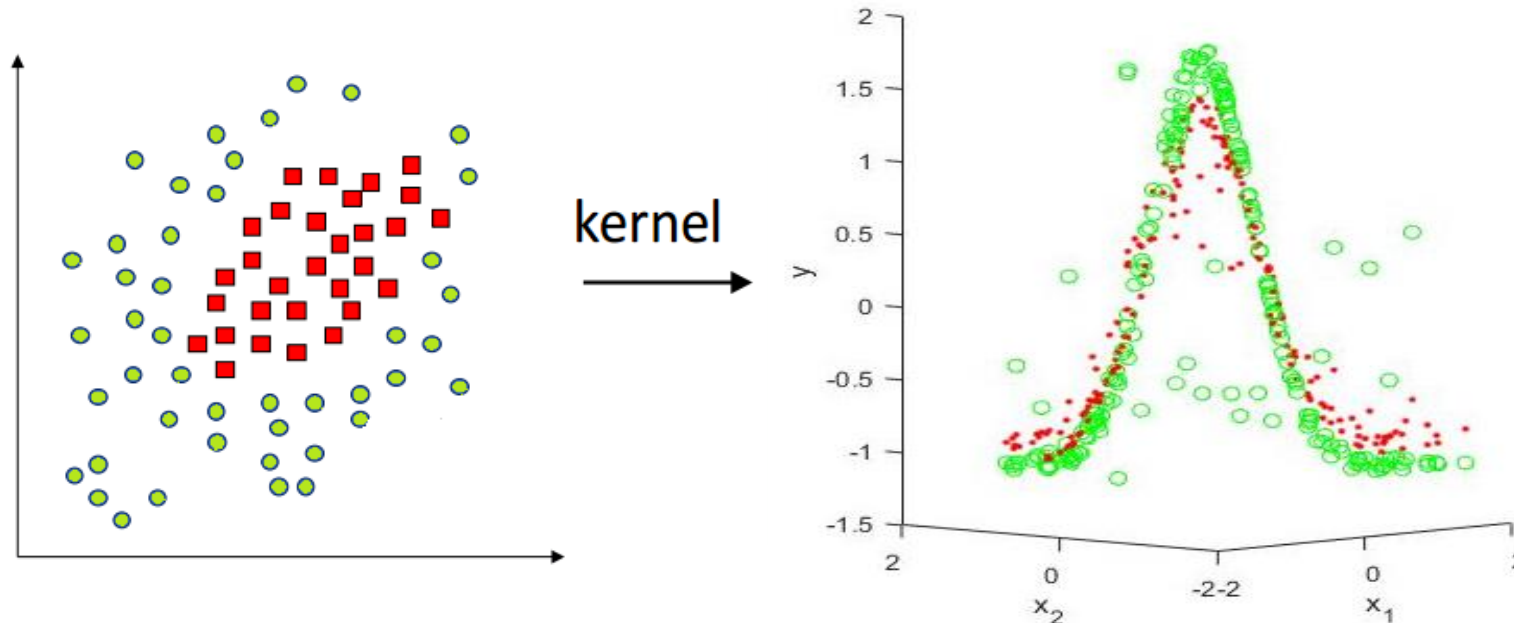
Regressão com SVM

- **Support Vector Regression (SVR)**
 - Assim como antes, é necessária a utilização de um “**Kernel trick**” para a aplicação da técnica em problemas **não lineares**.
 - Como vimos, esse truque **transforma** o conjunto de dados de maneira que o problema se torna **linear**:



Regressão com SVM

- **Support Vector Regression (SVR)**
 - A transformação envolve, por exemplo, adicionar mais uma **dimensão** aos dados do problema.
 - Com isso, os dados podem se tornar **ajustáveis** por um hiperplano.



Regressão com SVM

- **Support Vector Regression (SVR)**
 - Existem diferentes **tipos** de kernel, cada um mais adequado a um tipo de problema.
 - Alguns exemplos de Kernel:

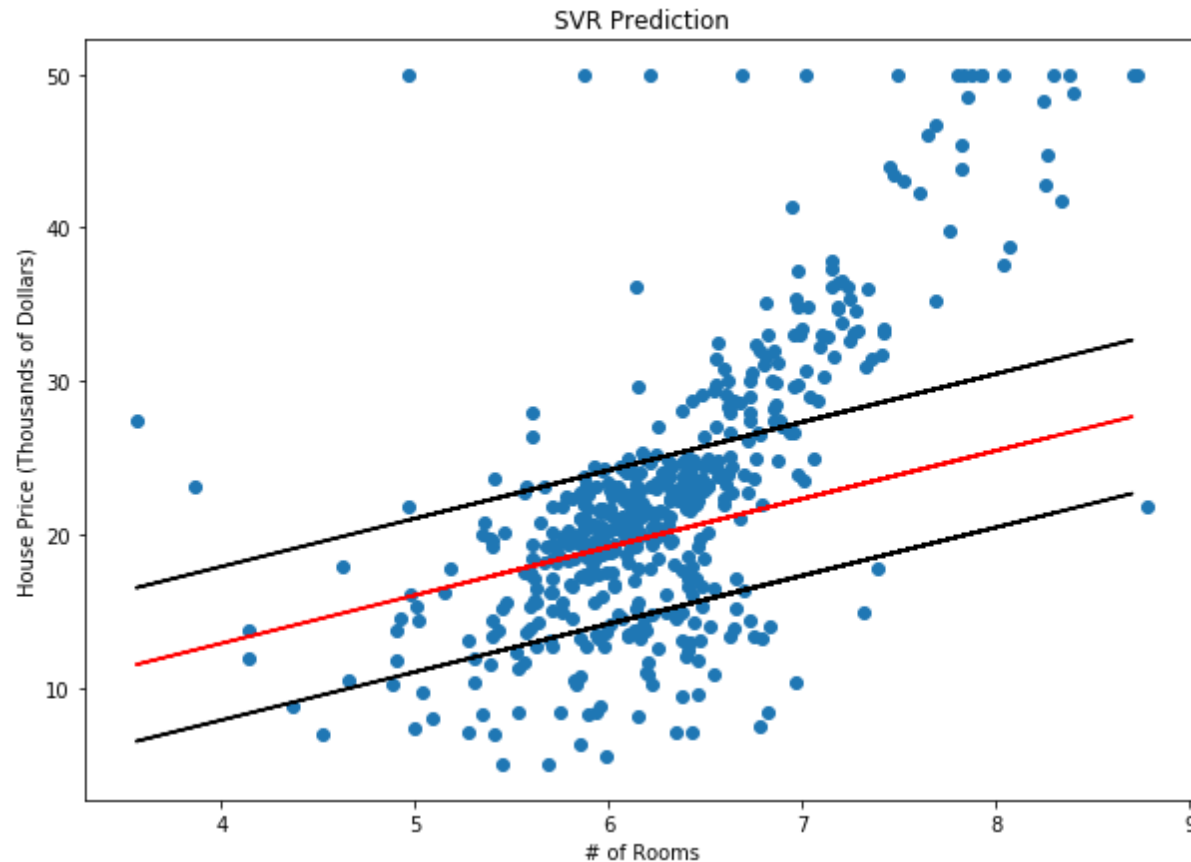
Kernels	Equation
1.Linear Kernel Function	$k(x_i, x_j) = 1 + x_i^T x_j$
2.Polynomial Kernel Function	$k(x_i, x_j) = (1 + x_i^T x_j)^p$
3.Radial Basis Kernel Function	$k(x_i, x_j) = \exp(-\gamma \ x_i - x_j\ ^2)$
4.Exponential radial basis kernel function	$k(x_i, x_j) = \exp(-\frac{\ x_i - x_j\ }{2\sigma^2})$
5.Gaussian Radial Basis Kernel Function	$k(x_i, x_j) = \exp(-\frac{\ x_i - x_j\ ^2}{2\sigma^2})$
6.Sigmoid Kernel Function	$k(x_i, x_j) = \tanh(kx_i^T x_j - \delta)$

- Tanto os resultados quanto o tempo de processamento podem variar de acordo com a escolha do kernel.

Regressão com SVM

- **Support Vector Regression (SVR)**
 - O SVR possui quatro parâmetros principais de configuração:
 - **Kernel**: escolher o tipo de kernel que será utilizado.
 - **Gamma**: determina a distância máxima que um elemento pode possuir a ponto de influenciar na construção dos kernels. Na prática, controla o tamanho dos kernels: quanto maior o valor de gamma, menor o raio do kernel.
 - **c**: punição por previsão incorreta. Um valor alto procura fazer o ajuste com 100% de acerto. Um valor mais baixo possui mais tolerância a erros, tornando o ajuste mais suave.
 - **Epsilon**: especifica a **margem de erro** que o ajuste pode considerar no ajuste da curva.

- **Support Vector Regression (SVR)**
 - Para facilitar o entendimento do novo parâmetro Epsilon (ϵ), vamos usar como exemplo o problema dos valores de imóveis.
 - Suponha que a única variável independente é o número de quartos.
 - Usando $\epsilon = 5$, temos o seguinte ajuste usando o SVR:



- **Support Vector Regression (SVR)**

- Vamos aplicar a técnica para a base com valores referentes ao preço de um plano de saúde de acordo com a idade do cliente:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

base = pd.read_csv('plano_saude.csv')

# Separando dados em previsores e classes
cols_previsores = ['idade']

cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Separando em base de testes e treinamento (usando 25% para teste)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                    objetivo,
                                                                                                    test_size=0.25,
                                                                                                    random_state=0)

# Visualização dos dados
plt.scatter(previsores, objetivo)
plt.title('Regressão linear (dados completos)')
plt.xlabel('idade')
plt.ylabel('custo')
```

- **Support Vector Regression (SVR)**

- Vamos aplicar a t
um plano de saúde

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np
```

```
##### Preprocessamento
```

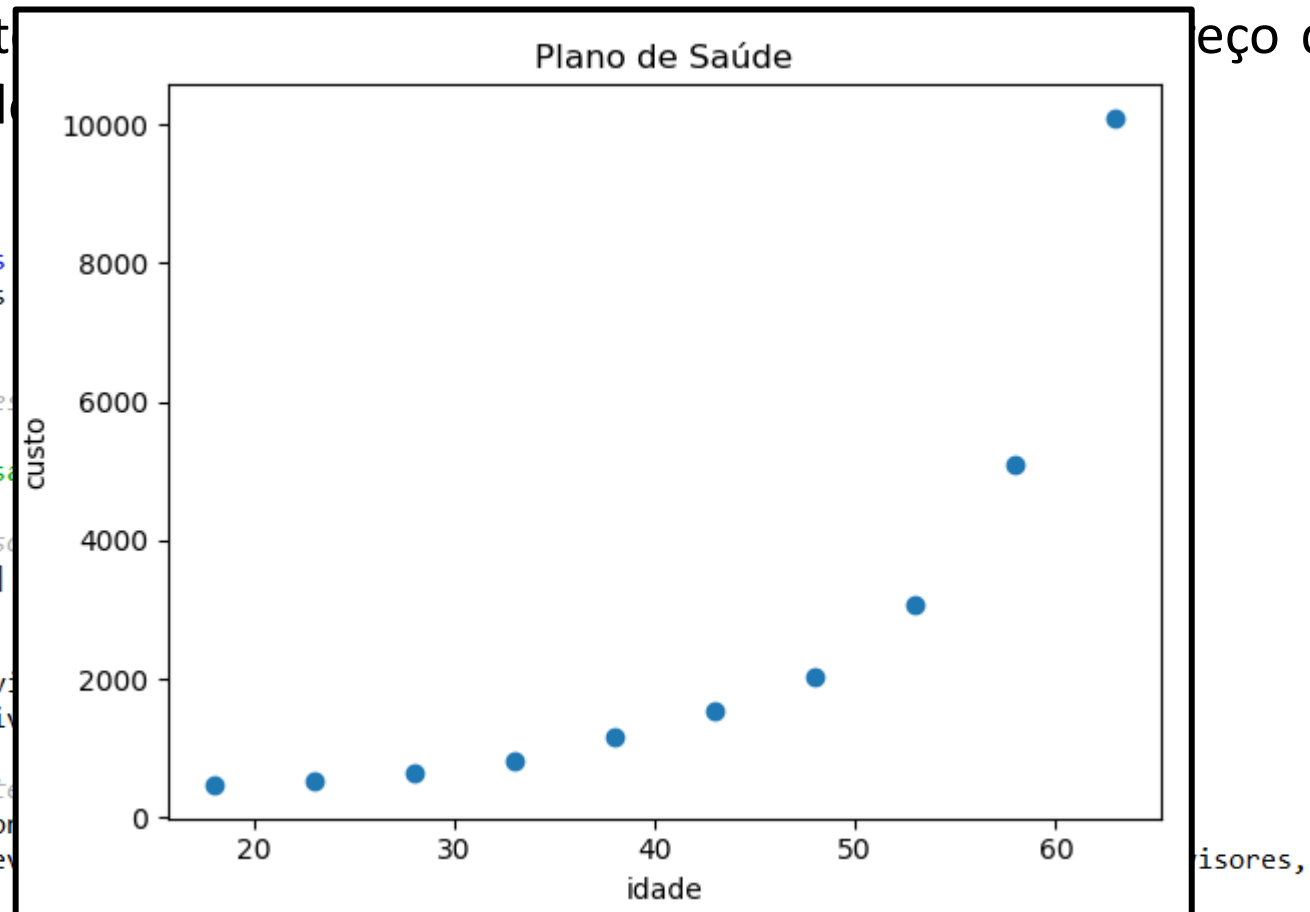
```
base = pd.read_csv('plano_saude.csv')
```

```
# Separando dados em previsores e objetivo
cols_previsores = ['idade']
```

```
cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]
```

```
# Separando em base de treinamento e teste
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores, objetivo, test_size=0.25, random_state=0)
```

```
# Visualização dos dados
plt.scatter(previsores, objetivo)
plt.title('Regressão linear (dados completos)')
plt.xlabel('idade')
plt.ylabel('custo')
```



test_size=0.25,
random_state=0)

- **Support Vector Regression (SVR)**

- Para aplicar a Regressão com SVM, devemos fazer:

```
##### Regressão com SVM #####
```

```
from sklearn.svm import SVR
regressor = SVR(kernel = 'rbf',
                 C = 1,
                 gamma = 'scale',
                 epsilon = 0.1)

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)
```

- **Support Vector Regression (SVR)**

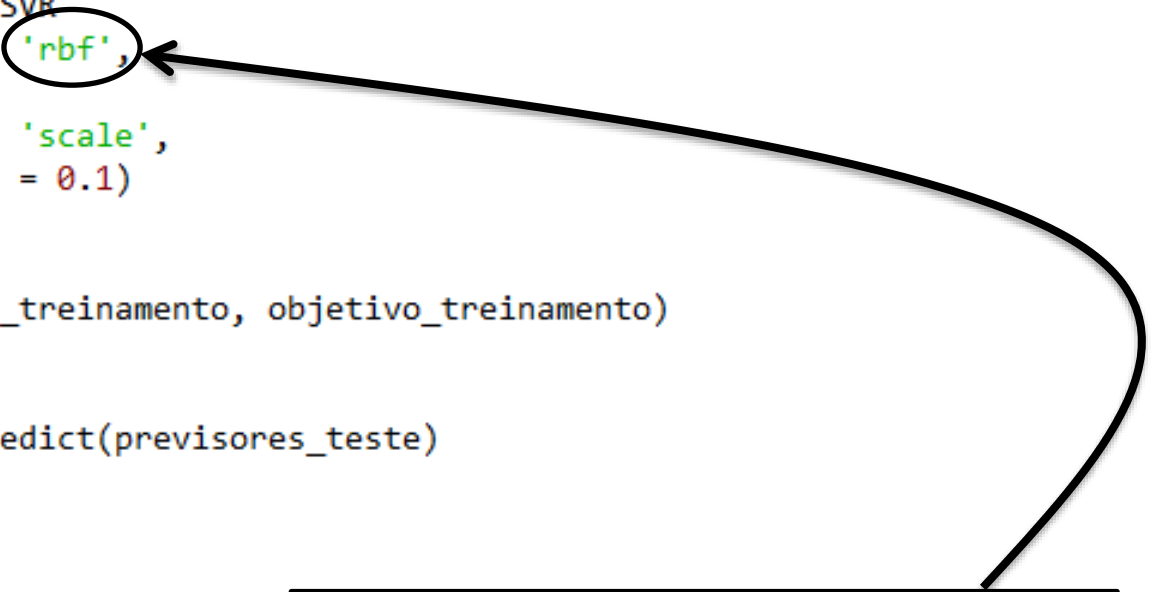
- Para aplicar a Regressão com SVM, devemos fazer:

```
##### Regressão com SVM #####
```

```
from sklearn.svm import SVR
regressor = SVR(kernel = 'rbf',
                  C = 1,
                  gamma = 'scale',
                  epsilon = 0.1)

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)
```



Parâmetro de configuração que corresponde ao kernel escolhido, no caso, o **radial basis function**, um dos mais utilizados.

- **Support Vector Regression (SVR)**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

```
##### Avaliação dos resultados #####

# Visualização dos dados de treinamento
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(previsores_treinamento, regressor.predict(previsores_treinamento), color = 'red')
plt.title('Regressão com SVM')
plt.xlabel('Idade')
plt.ylabel('Custo')

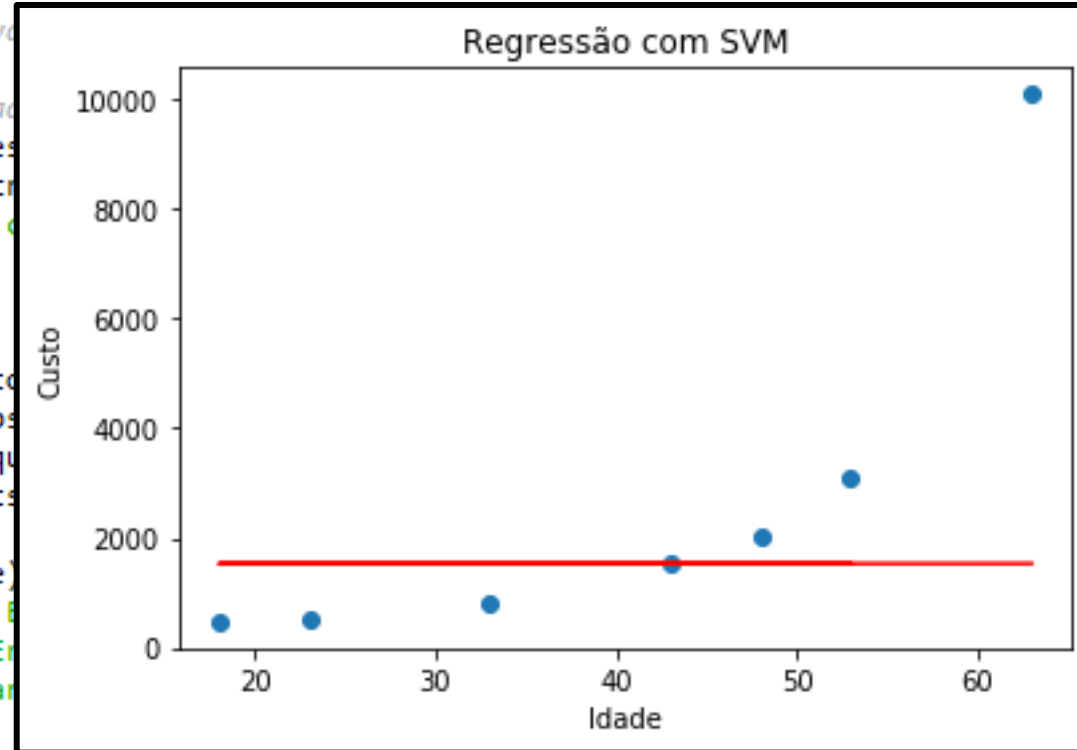
score = metrics.r2_score(objetivo_teste, previsoes)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

- **Support Vector Regression (SVR)**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

```
##### Avaliação do modelo #####  
  
# Visualização dos dados  
plt.scatter(previsores_teste, valores_teste)  
plt.plot(previsores_teste, y_hat, color='red')  
plt.title('Regressão com SVM')  
plt.xlabel('Idade')  
plt.ylabel('Custo')  
  
score = metrics.r2_score(y_hat, valores_teste)  
mae = metrics.mean_absolute_error(y_hat, valores_teste)  
mse = metrics.mean_squared_error(y_hat, valores_teste)  
rmse = np.sqrt(mse)  
  
print('Score:', score)  
print('Mean Absolute Error:', mae)  
print('Mean Squared Error:', mse)  
print('Root Mean Squared Error:', rmse)
```



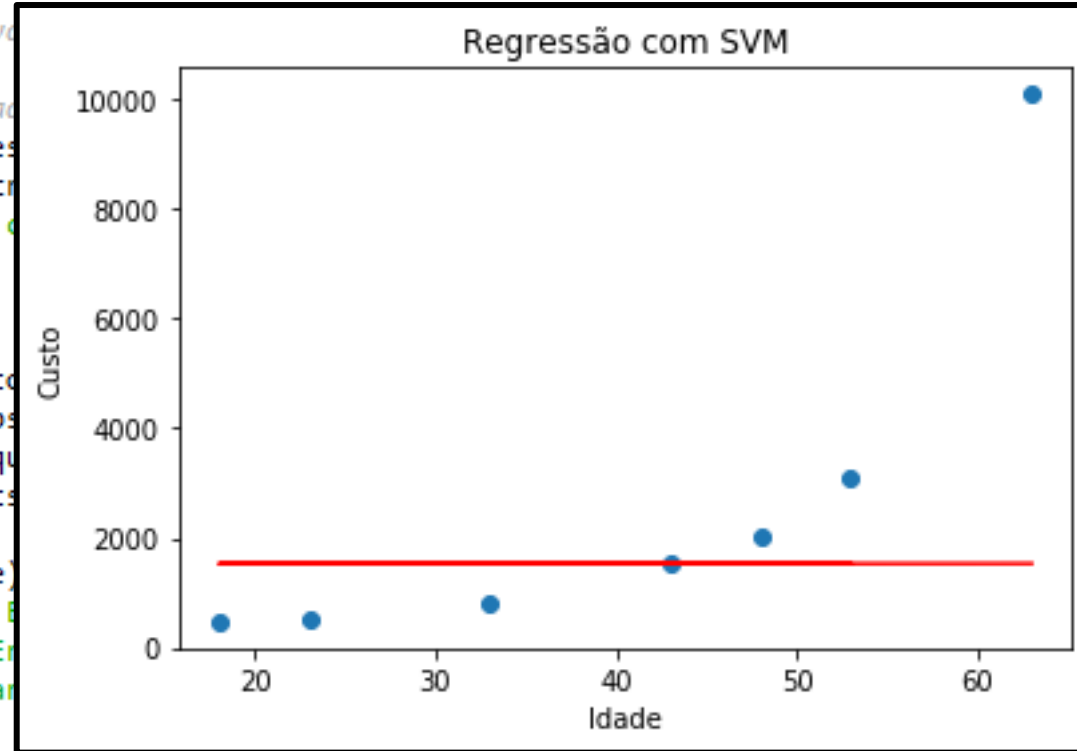
= 'red')

Perceba que o modelo treinado não teve boa aderência aos dados experimentais!

- **Support Vector Regression (SVR)**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

```
##### Avaliação do modelo #####  
  
# Visualização dos dados  
plt.scatter(previsores_teste, custo_teste)  
plt.plot(previsores_teste, y_hat, color='red')  
plt.title('Regressão com SVM')  
plt.xlabel('Idade')  
plt.ylabel('Custo')  
  
score = metrics.r2_score(y_hat, custo_teste)  
mae = metrics.mean_absolute_error(y_hat, custo_teste)  
mse = metrics.mean_squared_error(y_hat, custo_teste)  
rmse = np.sqrt(mse)  
  
print('Score:', score)  
print('Mean Absolute Error:', mae)  
print('Mean Squared Error:', mse)  
print('Root Mean Squared Error:', rmse)
```



Algumas técnicas, como o SVR, não trabalham bem com valores não padronizados.

Pode ser necessário padronizar tanto os previsores, quanto a variável objetivo.

Perceba que o modelo treinado não teve boa aderência aos dados experimentais!

- **Support Vector Regression (SVR)**

- Desta vez, vamos aplicar a padronização tanto nas variáveis previsoras quanto na variável objetivo:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

base = pd.read_csv('plano_saude.csv')

# Separando dados em previsores e classes
cols_previsores = ['idade']

cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Separando em base de testes e treinamento (usando 25% para teste)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                    objetivo,
                                                                                                    test_size=0.25,
                                                                                                    random_state=0)

# Padronização dos dados
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
previsores_treinamento = scaler.fit_transform(previsores_treinamento)
scaler_y = StandardScaler()
objetivo_treinamento = scaler_y.fit_transform(objetivo_treinamento)

# Visualização dos dados
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.title('Regressão com SVM (dados treinamento)')
plt.xlabel('idade')
plt.ylabel('custo')
```


- **Support Vector Regression (SVR)**

- Desta vez, vamos
quanto na variável

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np
```

```
##### Preprocessamento
```

```
base = pd.read_csv('plano_sa.csv')
```

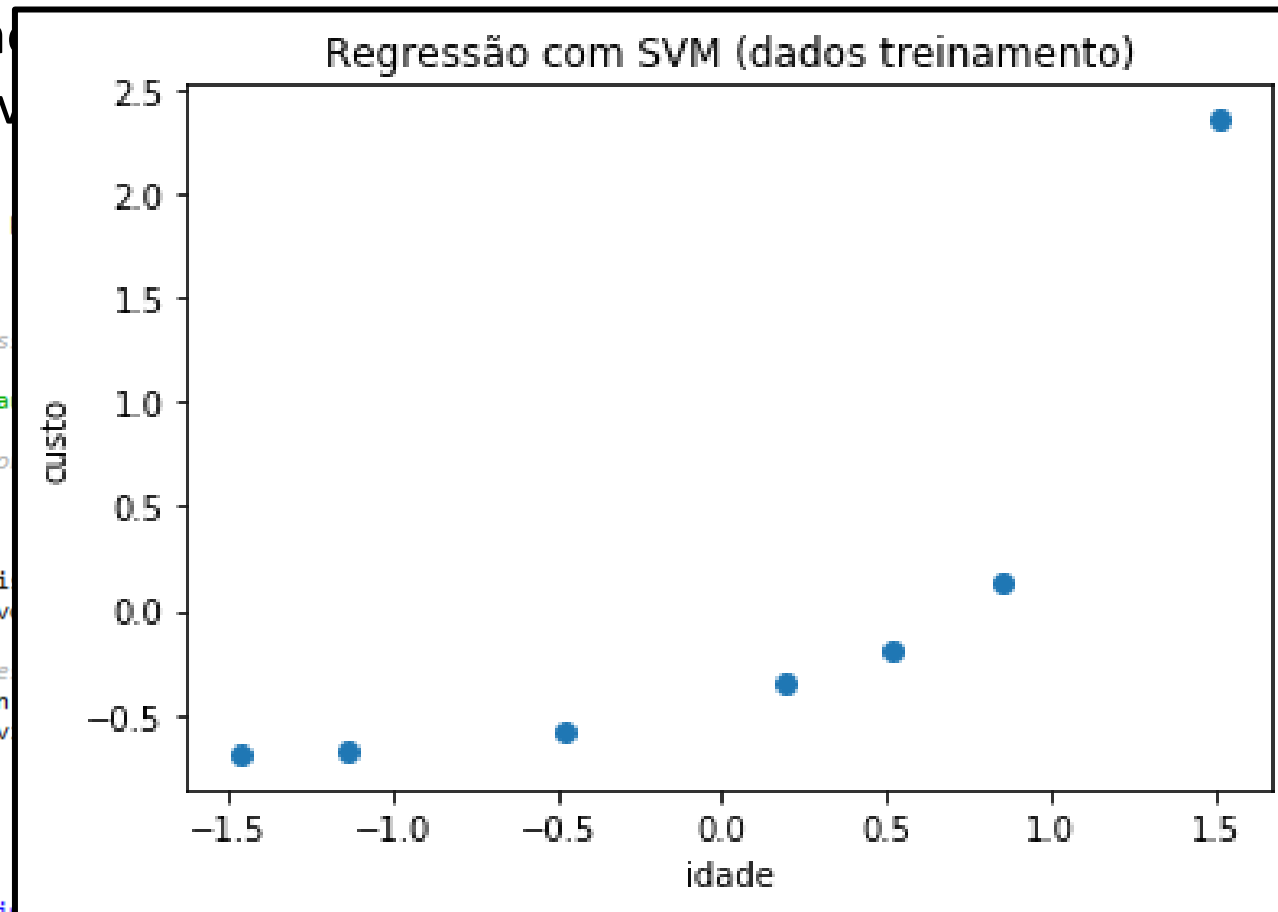
```
# Separando dados em preditores e objetivo
cols_preditores = ['idade']
```

```
cols_objetivo = ['custo']
preditores = base[cols_preditores]
objetivo = base[cols_objetivo]
```

```
# Separando em base de treinamento e teste
from sklearn.model_selection import train_test_split
preditores_treinamento, preditores_teste, objetivo_treinamento, objetivo_teste = train_test_split(preditores, objetivo, test_size=0.2, random_state=42)
```

```
# Padronização dos dados
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
preditores_treinamento = scaler.fit_transform(preditores_treinamento)
scaler_y = StandardScaler()
objetivo_treinamento = scaler_y.fit_transform(objetivo_treinamento)
```



Mesmo com a padronização, o comportamento dos dados permanece similar, mesmo que em outra escala.

- **Support Vector Regression (SVR)**

- Para aplicar a Regressão com SVM, devemos fazer:

```
##### Regressão com SVM #####
```

```
from sklearn.svm import SVR
regressor = SVR(kernel = 'rbf',
                 C = 1,
                 gamma = 'scale',
                 epsilon = 0.1)

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(scaler.transform(previsores_teste))
previsoes = scaler_y.inverse_transform(previsoes)
```

- **Support Vector Regression (SVR)**

- Para aplicar a Regressão com SVM, devemos fazer:

```
##### Regressão com SVM #####
```

```
from sklearn.svm import SVR
regressor = SVR(kernel = 'rbf',
                 C = 1,
                 gamma = 'scale',
                 epsilon = 0.1)
```

```
# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)
```

```
# Teste
previsoes = regressor.predict(scaler.transform(previsores_teste))
previsoes = scaler_y.inverse_transform(previsoes)
```

Nenhum parâmetro de configuração foi alterado.

Para ver a previsão na escala original, é necessário aplicar a transformação inversa da variável previsões, usando a mesma instancia do Scaler usado para a padronização da variável objetivo.

- **Support Vector Regression (SVR)**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

```
##### Avaliação dos resultados #####
```

```
# Visualização dos dados de treinamento
```

```
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(sorted(previsores_treinamento),
          sorted(regressor.predict(previsores_treinamento)), color = 'red')
plt.title('Regressão com SVM')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
# Visualização dos dados de teste
```

```
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(sorted(previsores_teste.values), sorted(previsoes), color = 'red')
plt.title('Regressão com SVM')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
score = metrics.r2_score(objetivo_teste, previsoes)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))
```

```
print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

- **Support Vector Regression (SVR)**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

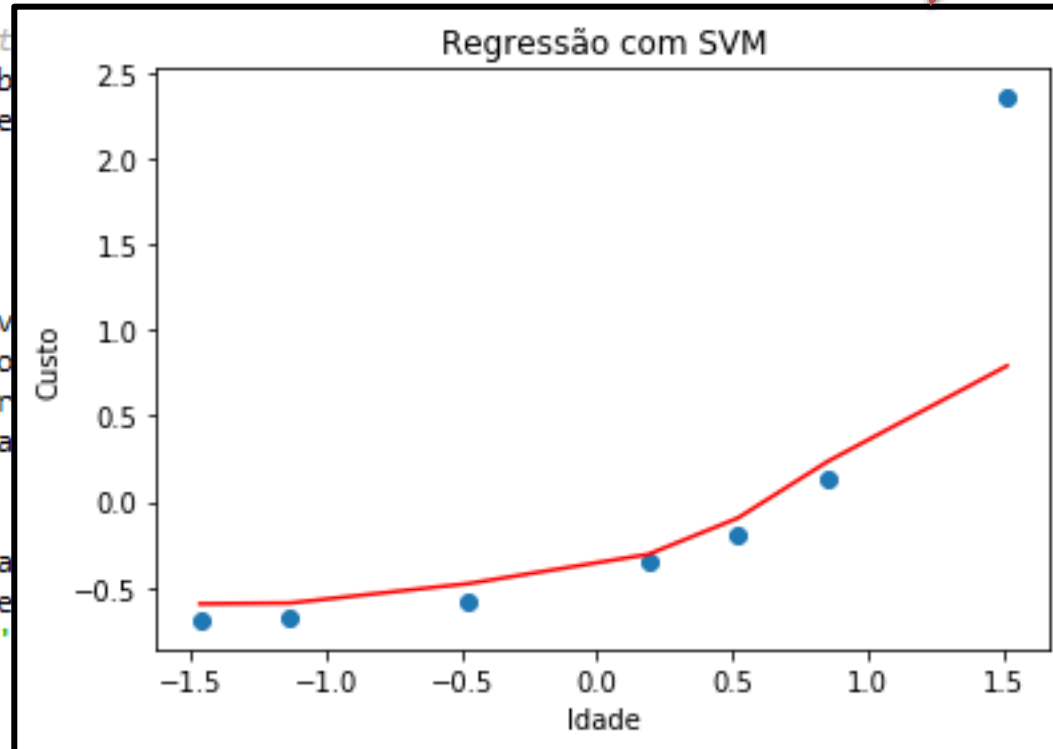
Avaliação dos resultados

```
# Visualização dos dados de treinamento
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(sorted(previsores_treinamento),
          sorted(regressor.predict(previsores_treinamento)), color = 'red')
plt.title('Regressão com SVM')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
# Visualização dos dados de teste
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(sorted(previsores_teste), sorted(regressor.predict(previsores_teste)), color = 'red')
plt.title('Regressão com SVM')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
score = metrics.r2_score(objetivo_teste, regressor.predict(previsores_teste))
mae = metrics.mean_absolute_error(objetivo_teste, regressor.predict(previsores_teste))
mse = metrics.mean_squared_error(objetivo_teste, regressor.predict(previsores_teste))
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, regressor.predict(previsores_teste)))
```

O modelo treinado se aproxima bastante dos dados experimentais, com exceção de um ponto.



- **Support Vector Regression (SVR)**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

```
##### Avaliação dos resultados #####
```

```
# Visualização dos dados de treinamento
```

```
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(sorted(previsores_treinamento),
          sorted(regressor.predict(previsores_treinamento)), color = 'red')
plt.title('Regressão com SVM')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

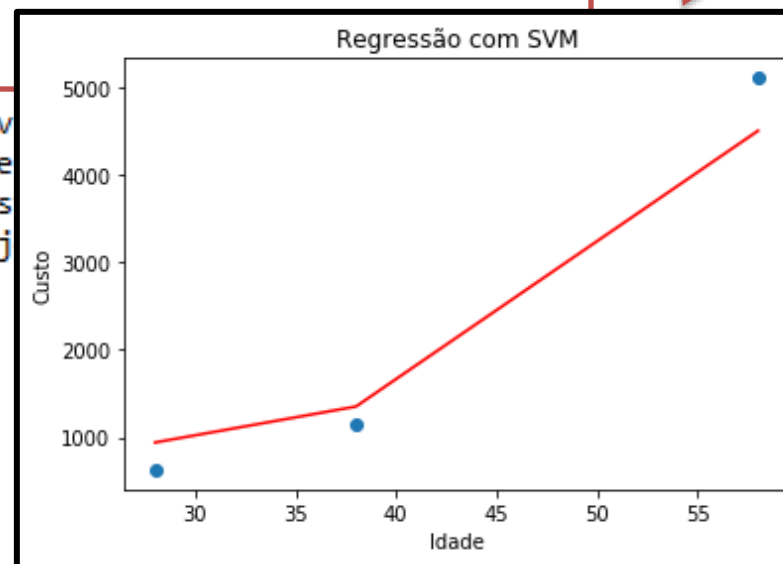
```
# Visualização dos dados de teste
```

```
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(sorted(previsores_teste.values), sorted(previsoes), color = 'red')
plt.title('Regressão com SVM')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
score = metrics.r2_score(objetivo_teste, prev
mae = metrics.mean_absolute_error(objetivo_te
mse = metrics.mean_squared_error(objetivo_tes
r(or(obj
```

Os resultados também foram
muitos bons na base de teste:

```
Score: 0.958120600984613
Mean Absolute Error: 371.49109526358296
Mean Squared Error: 166837.28849087545
Root Mean Squared Error: 408.4572052135639
```



- **Support Vector Regression (SVR)**

- Voltando ao problema dos preços de imóveis, o pré-processamento fica igual.

```
import pandas as pd

##### #Pré-processamento dos dados #####

#Leitura dos dados
base = pd.read_csv('house_prices.csv')
base.describe()

# Procurando valores inconsistentes
# Não há valores inconsistentes

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Separando dados em previsores e classes
cols_previsores = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
                   'floors', 'waterfront', 'view', 'condition', 'grade', 'sqft_above',
                   'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long']

# Não usarei: date, sqft_living15 e sqft_lot15

cols_objetivo = ['price']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Transforma as variáveis categóricas em valores numéricos
# Todas as variáveis são numéricas

# Padronização dos dados
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
previsores = scaler.fit_transform(previsores)

# Separando em base de testes e treinamento (usando 25% para teste)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
```

• Support Vector Regression (SVR)

- Para aplicar o SVR, ele deve ser definido como regressor, e seus parâmetros devem ser configurados.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Préprocessamento #####

scaler_y = StandardScaler()
objetivo_treinamento = scaler_y.fit_transform(objetivo_treinamento)

##### Regressão com Árvores de Decisão #####

from sklearn.svm import SVR
regressor = SVR(kernel = 'rbf',
                 C = 1,
                 gamma = 'scale',
                 epsilon = 0.1)

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)
previsoes = scaler_y.inverse_transform(previsoes)

##### Avaliação dos resultados #####

score = metrics.r2_score(objetivo_teste, previsoes)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

Em alguns casos, pode ser interessante incluir a variável objetivo na padronização dos dados.

Porém, quando isso for feito, é bom lembrar de desfazer a padronização antes de calcular as métricas.

• Support Vector

- Para aplicação de parâmetros

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import linear_model
import numpy as np
```

```
##### Préprocessamento #####
```

```
scaler_y = StandardScaler()
objetivo_treino = scaler_y.fit_transform(y_train)
```

```
#####
```

```
from sklearn.metrics import r2_score
regressor = SVR()
```

```
# Treinamento
regressor.fit(X_train, objetivo_treino)
```

```
# Teste
previsoes = regressor.predict(X_test)
```

```
previsoes = scaler_y.inverse_transform(previsoes)
```

```
#####
```

```
score = r2_score(y_test, previsoes)
mae = metrics.mean_absolute_error(y_test, previsoes)
mse = metrics.mean_squared_error(y_test, previsoes)
rmse = np.sqrt(mse)
```

```
print('Score: %f' % score)
print('Mean Absolute Error: %f' % mae)
print('Mean Squared Error: %f' % mse)
print('Root Mean Squared Error: %f' % rmse)
```

Vamos registrar os resultados em um arquivo único, para que possamos compará-lo com resultados futuros:

```
Score: 0.7640237636433408
Mean Absolute Error: 81127.38528255295
Mean Squared Error: 31347221961.071903
Root Mean Squared Error: 177051.46698367654
```

na padronização dos dados.

House Sales in King County, USA				
Descrição		Dados e imóveis e seus respectivos valores em um distrito dos Estados Unidos.		
Total de registros:		21613		
Total de características:		16		
Total de colunas objetivo:		1		
Resultados				
Método	Configuração	Tratamento de dados	Score %	RMSE
Regressão Linear	-	Padronização	0.69	202633.35
Regressão Polinomial	Grau 2	Padronização	0.8083	159537.82
Árvores de Decisão	max_depth=9	Padronização	0.8002	162896.46
Random Forest	n_estimators=100, max_features=10, max_depth=15	Padronização	0.8889	121506.52
SVR	kernel = 'rbf', C = 1, gamma = 'scale', epsilon = 0.1	Padronização	0.7640	177051.47

regressor, e seus

pode ser
variável objetivo

feito, é
per a
cular as

Regressão com SVM

FIM